

思考了 43s

是的，而且我看完这两篇之后，反而觉得它们很适合拿来帮你把第二篇的研究边界讲清楚。

但有一个很关键的区别：

这两篇工作的共同思想确实都是“不能完全相信 LLM 生成出来的中间结果”，但它们主要怀疑的是不可执行、执行失败、检索路径错误或结果不合理；而你的工作要进一步怀疑：即使 Python 成功执行并返回了一个看起来合理的结果，也不能认为代码正确实现了问题语义。

这个差异非常重要。尤其第一篇 ARI-KBQA，某种意义上甚至可以成为你提出 **Execution-Trust Bias** 的一个很好的对照案例。

一、先说结论：这两篇和你像，但没有把你的核心问题解决掉

我会把三者这样区分：

方法	它“不信任”什么	主要检查什么	成功执行后还审计代码语义吗？
ARI-KBQA	LLM 生成的 Logical Form	LF 是否可执行、关系路径是否可行	基本没有
Code-on-Graph	LLM 规划、代码、执行过程	execution error、empty result、answer plausibility	有一点，但很粗
你的方法	Python 成功执行后的结果	Question Intent 与 Code Intent 是否多粒度一致	是核心问题

所以我认为你现在的论文故事可以比之前更明确：

Previous work asks: **Can the generated program be executed?**
We ask: **Even if it executes successfully, did it execute the right semantics?**

甚至可以浓缩成一句很适合论文 Introduction 的话：

Executability is not semantic faithfulness.

这句话非常适合你第二篇。

二、第一篇 ARI-KBQA 和你哪里像？

第一篇是：

Generating then Refining for Reliable Knowledge Base Question Answering / ARI-KBQA

它的整体思路与你第一篇和第二篇都有明显相似性：

Generate → Verify → Refine

论文设计了 Action Generator 和 Reflection Verifier。Generator 一步一步产生 logical form，Verifier 与知识库交互，判断当前路径是否可行，如果发现错误，就给 Generator semantic correction feedback，再继续修改。

它第 4 页 Figure 3 的图也非常直观：

Action Generator → 当前 LF → Reflection Verifier → KB → Replace Error LF → 回到 Generator

这一点你可以直接借鉴。

你的版本可以变成：

Code Generator → Executable Python → Semantic Auditor → Semantic Logic Exception → Targeted Repair → Code Generator

所以闭环结构可以借。

三、但 ARI-KBQA 有一个特别关键的“漏洞”，恰好就是你的研究点

这是我觉得两篇里面对你最有价值的一处。

ARI-KBQA 明确写道：

如果当前 hop 存在一个 **executable LF**，它就被认为与事实一致并具有较高准确率，然后直接进入下一 hop；只有无法执行的时候，才调用 Reflection Verifier。

最终阶段也一样：

如果存在 executable RLF，答案就直接来自这个 LF 的 **execution result**。

这实际上恰好体现了你说的那个思想：

Executable → Trusted

虽然作者没有把它称作 Execution-Trust Bias。

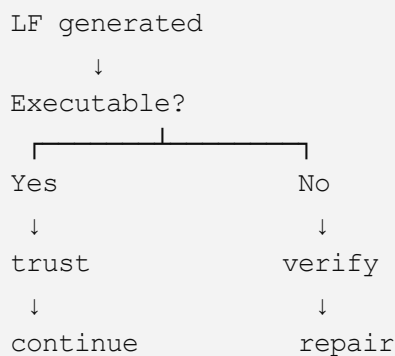
他们的主要实验动机也是：non-executable logical forms 导致性能严重下降，因此重点是提高 LF 的 executability。 2ae2634d-ce3d-4201-b101-2a032d3...

所以这篇论文与你并不是“撞车”。

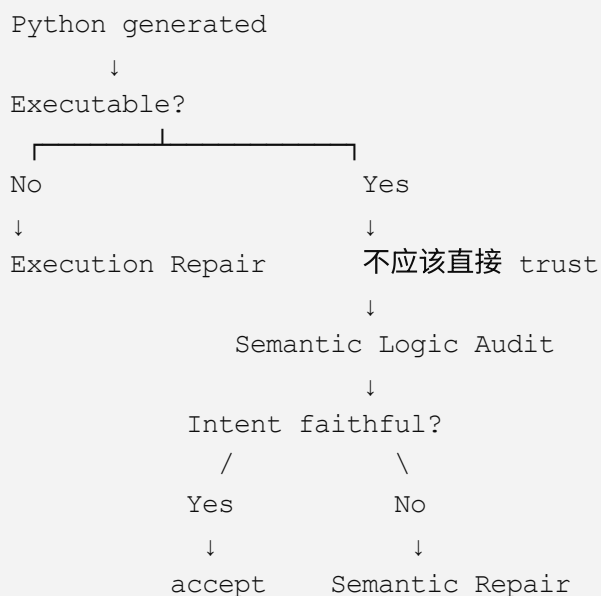
反而你可以把它看成：

你研究的问题恰好位于 **ARI-KBQA** 验证边界之后。

ARI:



而你的观点是：



这就是非常清楚的研究差异。

四、所以 ARI-KBQA 最值得你借的不是“可执行检查”，而是“局部验证”

ARI-KBQA 的一个很好设计是：

不是等最终 LF 全生成完再看，而是 hop-by-hop 验证。

Reflection Verifier 会拿：

- Question;
- topic entity;
- previous LF;
- 当前候选 relation;

然后验证这一 hop 的 relation 是否正确，并纠正错误 relation。

2ae2634d-ce3d-4201-b101-2a032d3...

这个思想非常适合转移到你的 **Operator-level Audit**。

比如问题：

What is the total number of UCI ProTour points scored by an Italian cyclist?

Python：

Python

运行

```
italian = df[df["Cyclist"].str.contains("ITA")]  
answer = italian["UCI ProTour\nPoints"].mean()
```

不要只把整个 Python 当成一个黑盒。

可以拆成：

```
OP1:  
FILTER  
Cyclist contains ITA  
  
OP2:  
SELECT  
UCI ProTour Points  
  
OP3:  
AGGREGATE  
mean
```

Question Intent:

```
OP1:
FILTER
nationality = Italy
```

```
OP2:
SELECT
UCI ProTour Points
```

```
OP3:
AGGREGATE
sum
```

然后进行类似 ARI 的 **operation-by-operation alignment**:

```
OP1 ✓
OP2 ✓
OP3 ✗
```

```
Semantic Logic Exception:
OperatorMismatch
```

```
Expected:
SUM
```

```
Actual:
MEAN
```

这比单纯:

“Analyst, 请判断代码对不对。”

强很多。

也更符合你老师说的**多粒度思想**。

五、甚至可以把你的三级结构进一步升级

你之前设计的是:

Level 1

Global Intent Alignment

Level 2

Operator Logic Alignment

Level 3

Parameter & Entity Alignment

这个已经可以。

但读完 ARI 后，我建议把第二层内部再做成**序列化 operation alignment**：

Question Intent Graph / Sequence

```
FILTER(region = East)
    ↓
FILTER(year > 2022)
    ↓
SELECT(sales)
    ↓
SUM
    ↓
NUMBER
```

Code Intent:

```
FILTER(region = East)
    ↓
SELECT(sales)
    ↓
SUM
    ↓
NUMBER
```

然后不是只比较：

```
aggregation = sum
```

而是比较整个：

```
Intent Operation Sequence
    VS
Code Operation Sequence
```

得到：

```
Missing Operation:
FILTER(year > 2022)
```

这个设计明显比简单 JSON field comparison 更像一个完整方法。

六、第二篇 Code-on-Graph 跟你更像

第二篇：

Code-on-Graph: Iterative Programmatic Reasoning via Large Language Models on Knowledge Graphs

这个和你的第二篇更接近，因为它真的：

生成 Python → 执行 Python → 根据执行反馈修代码

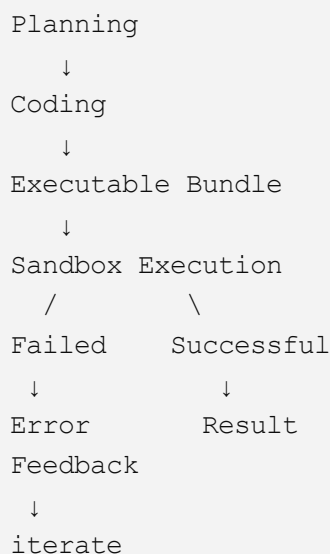
它把整个过程分成三部分：

1. Planning
2. Coding
3. Executing

并将 KG schema 转换成 Python class，再生成动态 Python 代码进行 reasoning。

a7d82e1e-0f81-48a9-bb9c-22b97ae...

它第 3 页 Figure 2 里的结构尤其值得你看：



这个结构与你现在的 Python Agent 非常接近。

七、Code-on-Graph 已经碰到了你的一部分问题

这篇比 ARI 更接近你，因为它已经意识到：

“没有 Syntax Error”还不完全够。

例如它特别规定：

如果执行结果是 **empty result**，虽然 Python 本身没有抛 exception，它仍然把它视为 failure，因为 empty output 往往代表生成代码和 KG constraint 之间存在逻辑不匹配。

```
a7d82e1e-0f81-48a9-bb9c-22b97ae...
```

这个思想非常值得借。

也就是：

```
Runtime Success
≠
Task Success
```

这实际上已经向你的 Execution-Trust Bias 靠近了一步。

但它只走到：

```
Exception      → suspicious
Empty Result   → suspicious
Valid Result   → basically accepted
```

你的工作继续走：

```
Exception      → Execution Error
Empty Result   → Semantic Warning
Non-empty Result
↓
仍然 Semantic Audit
```

所以你可以把 CoG 看成：

Execution-aware verification

你的则是：

Execution-success semantic verification

八、Code-on-Graph 还有一个地方和你的 Global Intent Alignment 几乎对应

这个特别值得借。

CoG 在 Planning 阶段有一个 Evaluator。

Evaluator 决定是否继续推理时，会检查两个方面：

1. Exploration Sufficiency

问题中的关键 constraint 是否已经被覆盖。

论文甚至明确举：

- temporal qualifiers;
- superlatives。

2. Answer Plausibility

当前执行结果是否：

- 类型正确；
- 与预期答案形式语义一致；
- 比如 entity / number / date。

a7d82e1e-0f81-48a9-bb9c-22b97ae...

你看，这基本就是你现在的：

Global Intent Alignment

例如：

Question:

Who was ranked next after Davide Rebellin?

Expected:

entity

Python:

```
df["Rank"].sum()
```

Result:

55

Global Auditor:

Expected answer type:

ENTITY

Observed:

NUMBER

GLOBAL_TYPE_MISMATCH

所以这个设计是完全可以吸收的。

不过你要往前走一步：

CoG 主要看：

Result ↔ Expected Answer Form

你的：

Question Intent

↓

Code Intent

↓

Execution Result

三者一起检查。

九、Code-on-Graph 的 Error Analysis 甚至在替你证明问题存在

这可能是第二篇论文里对你最有价值的地方。

他们专门分析了一类：

Reasoning Error

论文说：

即使 CoG 已经 recall 了正确 schema，仍然可能因为 **semantic misunderstanding** 或者选择了“相似但是错误的 relation”，最终得到错误答案。

a7d82e1e-0f81-48a9-bb9c-22b97ae...

例如：

正确路径应该是：

```
pro_athlete.teams  
→ sports_team_roster.team
```

但模型写成：

```
player_statistics  
→ player_stats_team
```

最终得到了错误的球队。

注意，这已经不是：

```
Syntax Error  
AttributeError
```

而是：

程序表达的语义错了。

这和你的：

```
Question asks SUM
Python uses MEAN
```

本质非常接近。

区别是：

CoG 把它放在最后的 **Error Analysis** 中作为仍然存在的失败类型；

而你要把它提升成：

Method 本身要主动检测的研究对象。

这非常关键。

换句话说：

CoG：

我们发现还有 semantic reasoning errors。

你：

我专门提出机制，在 execution success 后检测这种 semantic reasoning error。

这个关系很漂亮。

十、这里甚至可以帮你重新定义 Execution-Trust Bias

我建议不要只写：

“模型认为代码跑通了就是答案正确。”

可以定义得再学术一点：

Execution-Trust Bias describes the tendency of tool-augmented reasoning systems to use executability or successful runtime completion as an implicit proxy for semantic faithfulness.

中文就是：

执行信任偏差是指工具增强推理系统倾向于将程序的可执行性或成功运行，隐式地视为程序忠实实现原问题语义的证据。

这比单纯：

code runs = correct

要完整很多。

因为它直接可以覆盖：

- ARI 的 executable LF；
- CoG 的 valid execution result；
- ReAcTable 的 Python execution；
- 你后面其他 program-aided TableQA baseline。

十一、这两篇最大的共同点，其实不是“不相信最终答案”

这里我要帮你稍微修正一下表述。

你说：

“是不是都是不信任结果？”

大方向是，但论文里最好不要这么写。

准确来说：

ARI-KBQA

不信任的是：

LLM 生成的 logical form / relation path

尤其是 non-executable LF。

Code-on-Graph

不信任的是：

LLM 的 planning + coding + execution 过程

它至少会处理：

- execution exception；

- empty result;
- result plausibility。

你的工作

不信任的是：

successful execution 所产生的“正确性错觉”。

因此三者可以放在一个验证演进关系：

```
Generation Reliability
      ↓
Execution Reliability
      ↓
Semantic Faithfulness
```

或者：

```
Can it be generated?
      ↓
Can it be executed?
      ↓
Does it execute the intended semantics?
```

你的工作在第三层。

十二、我认为你最应该从 ARI 借这 3 个东西

1. Generate–Verify–Refine 闭环

你的名字可以不一样，但结构可以吸收：

```
Generate
      ↓
Execute
      ↓
Audit
      ↓
Refine
```

而且你比 ARI 多一个 Execute：

```
Generate → Execute → Semantic Verify → Refine
```

非常自然。

2. 局部错误定位，而不是一句“代码错了”

ARI 的 Verifier 是找：

哪个 relation 有问题。

你应该找：

哪个 semantic operation 有问题。

例如：

```
Error Level:
Operator

Error Location:
Operation 3

Expected:
SUM(UCI Points)

Actual:
MEAN(UCI Points)
```

而不是：

```
The code may be incorrect.
```

这会直接加强你论文的 interpretability。

3. Dynamic Verifier

ARI 为了效率，不是所有时候都调用 verifier；它的 verifier 在大量步骤中处于 idle 状态。论文还专门报告了这一点。

```
2ae2634d-ce3d-4201-b101-2a032d3...
```

但这里你不能照搬它“**success** 就不 **audit**”。

你的设计可以改成：

Cheap Audit 永远运行

```
AST
schema grounding
answer type
operator extraction
```

这些不需要额外 LLM API。

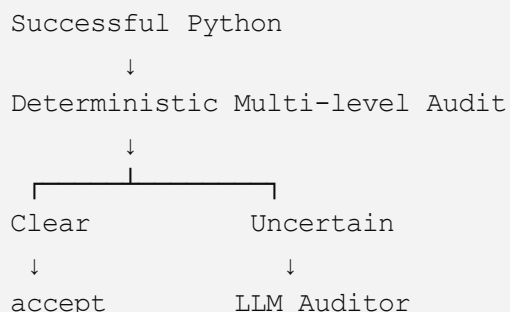
Expensive Semantic Auditor 按需调用

只有：

```
IR ambiguity
entity alias ambiguity
low-confidence alignment
rule conflict
```

才调用 DeepSeek。

于是：



这样同时保证：

- 论文理念：成功也审计；
- API 成本：不是每个 case 都多调很多次。

我觉得这会让你原来“Python 每次都再叫一个 Analyst”更好。

十三、Code-on-Graph 最值得你借的有 4 个

第一：Schema grounding

CoG 的一个核心贡献是把 KG schema 转成 Python class，使推理不是面对一堆原始 triples，而是面对明确的 typed structure。它强调这种 schema abstraction 可以支持 traversal、filtering、sorting、aggregation 和 constraint checking。

```
a7d82e1e-0f81-48a9-bb9c-22b97ae...
```

你不需要照搬 Python Class。

但是可以建立：

Table Semantic Schema

例如你刚才给我的 WTQ 表：

JSON

```
{
  "Rank": {
    "type": "integer",
    "role": "ordinal"
  },
  "Cyclist": {
    "type": "entity",
    "contains_country_code": true
  },
  "Team": {
    "type": "entity"
  },
  "Time": {
    "type": "duration"
  },
  "UCI ProTour Points": {
    "type": "number",
    "role": "measure"
  }
}
```

然后 Parameter Auditor 不只是：

字符串比较列名

而是：

```
Question target semantic role
      ↓
Table schema
      ↓
Code selected column
```

这个会明显提升细粒度审计。

十四、第二：Execution Trace

CoG 保存：

```
(T1, R1)
(T2, R2)
...
```


也就是 subtask + execution result 的 history。

你也可以做：

```
Operation 1:  
Filter nationality == ITA  
Rows: 10 → 3
```

```
Operation 2:  
Select UCI Points  
Values: [25,20,15]
```

```
Operation 3:  
SUM  
Output: 60
```

这叫：

Semantic Execution Trace

我甚至觉得这个名字很好。

这样 Code Intent 不只来自 AST：

```
Static Intent
```

还来自 runtime：

```
Dynamic Intent
```

最终：

```
Code Intent IR  
=  
Static Program Semantics  
+  
Dynamic Execution Trace
```

这个比“让 Analyst 读 Python 然后翻译”强得多。

十五、第三：Empty Result 作为异常信号

这个直接拿来。

你可以设计：

包括：

```
empty dataframe
empty answer
unexpected scalar
unexpected multi-answer
filter removes all rows
aggregation over empty set
NaN
None
```

这些虽然不是 Python exception，但都应该：

```
semantic_warning = true
```

这是很实用的 deterministic layer。

十六、第四：有限 Repair 次数

CoG 会限制 retry 次数，而且实验发现增加 correction attempt 能提高性能，但同时增加 token/time cost，因此推荐有限的 retry budget。

a7d82e1e-0f81-48a9-bb9c-22b97ae...

这与你第一篇 K=3 的故事也很搭。

你的第二篇可以设置：

```
Max Semantic Repair = 2
```

或者实验：

```
0
1
2
3
```

然后报告：

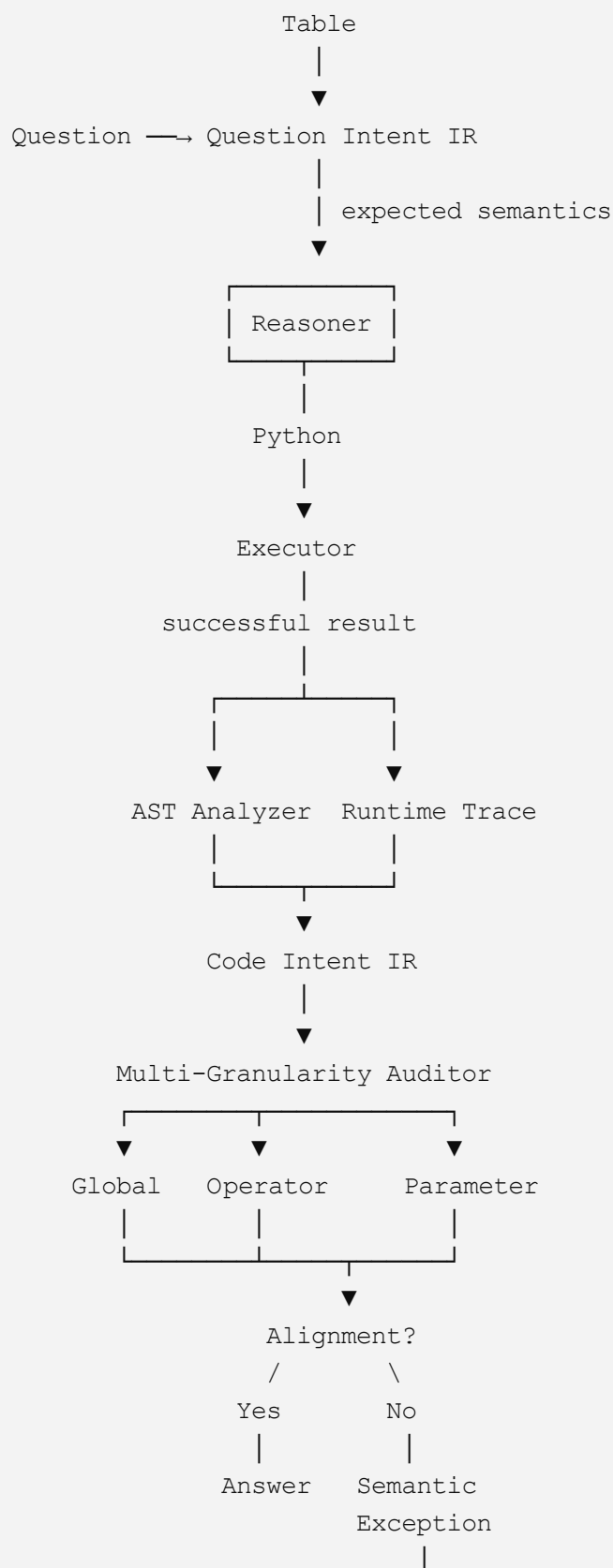
```
Accuracy
Repair Success
Token Cost
```

很容易形成一张分析图。

十七、我现在反而建议把你的方法变成“四步”，而不是只强调 Analyst

综合这两篇之后，我建议核心架构升级为：

Question-Code Semantic Consistency Auditing



这个结构已经明显比：

Analyst 看代码，然后判断一下。

强很多。

十八、还有一个非常值得增加的概念：Static + Dynamic 双视角

这个我觉得你尤其应该加。

你原来的 Code Intent IR 主要来自：

AST 逆向解析。

现在结合 Code-on-Graph，可以形成：

Static Semantic View

代码打算做什么：

Python

运行

```
df[df.year > 2020]["sales"].sum()
```

AST：

```
FILTER year > 2020
SELECT sales
SUM
```

Dynamic Semantic View

代码实际上对什么数据做了什么：

```
Input rows = 10
Filter year > 2020
Remaining rows = [2021, 2022, 2023]
Selected sales = [...]
Output = 532
```

最终：

Question Intent IR
vs
Static Code Intent IR
vs
Dynamic Execution Intent IR

这就变成一个三方一致性验证。

这个我认为比你最初只有：

Question IR vs Code IR

还要好。

十九、这样你的“多粒度”就不只是老师要求加进去的装饰

最终可以形成一个很漂亮的二维框架：

一维是粒度：

Global
Operator
Parameter

另一维是证据来源：

Question Semantics
Static Code Semantics
Dynamic Execution Trace
Table Grounding

相当于：

	Global	Operator	Parameter
Question	answer type	required operations	entity/value
Static AST	return type	actual operators	column/constant
Runtime	actual result type	execution sequence	actual rows/cells
Table grounding	answer feasibility	schema validity	cell consistency

这个框架就一下子立起来了。

我认为这个比单纯三级审计更像一篇完整的方法论文。

二十、而且特别适合做消融

以后实验可以直接做：

Granularity Ablation

G
G + O
G + O + P

Evidence Ablation

LLM-only Audit
AST-only
AST + Schema
AST + Runtime
AST + Runtime + Schema + LLM

Execution-Aware Ablation

No Audit
Failure-only Audit
Result Plausibility Audit
Full Semantic Audit

最后这个尤其漂亮。

因为它实际上可以模拟：

ReAcTable-like
CoG-like
Ours

虽然论文里不要随便说就是它们本身，只说 verification depth 的不同。

二十一、现在可以形成一条很强的“验证能力演进线”

你可以把 Related Work / Introduction 中的方法差异画成：

Level 0

No Verification
↓
Level 1
Execution Error Verification
Syntax / Runtime Error
↓
Level 2
Execution Feasibility Verification
Executable / Empty / Invalid Path
↓
Level 3
Answer Plausibility Verification
Type / Cardinality
↓
Level 4
Semantic Faithfulness Verification ← Ours
Global + Operator + Parameter

其中：

ARI

主要处于：

Level 2

CoG

已经碰到：

Level 2 + 部分 Level 3

你的方法

核心就是：

Level 4

这样“多粒度”甚至可以和“验证深度”联系起来。

二十二、不过有一个地方你要特别小心：CoG 已经有“semantic aligned”这句话

这个必须注意。

因为 CoG 的 evaluator 明确说：

检查 result 是否与 expected answer form **semantically aligned**。

a7d82e1e-0f81-48a9-bb9c-22b97ae...

所以你未来写论文的时候不能简单写：

“Existing methods only detect syntax/runtime errors and do not perform any semantic verification.”

这句话会被审稿人抓住。

应该写得更精确：

Existing execution-aware frameworks may assess execution validity, empty outputs, or coarse answer plausibility, yet they do not explicitly reconstruct and align the semantics of successfully executed programs with the original question at multiple granularities.

这样就安全得多。

也准确得多。

二十三、甚至 CoG 的失败案例可以直接启发你构造 Semantic Bug Benchmark

它把错误分成：

- Subtask Planning Error
- Retrieval Error
- Reasoning Error
- Max Attempts Error

a7d82e1e-0f81-48a9-bb9c-22b97ae...

你可以针对 TableQA 进一步专门定义：

Semantic Logic Error Taxonomy

Global

- Answer Type Error
- Cardinality Error
- Task Intent Error

Operator

- Missing Filter
- Extra Filter
- Wrong Aggregation
- Wrong Ranking

- Wrong Arithmetic
- Missing Comparison

Parameter

- Wrong Column
- Wrong Entity
- Wrong Constant
- Wrong Comparator
- Wrong Temporal Boundary
- Wrong Cell

这个 taxonomy 将来本身都可以是一张论文图或者表。

二十四、我觉得你现在最值得加的一个实验：Controlled Semantic Mutation

这个思路结合 ARI 的 verifier training 和 CoG error taxonomy 后特别适合你。

拿一段本来正确的 Python：

Python

运行

```
df[df["Cyclist"].str.contains("ITA")]["UCI ProTour_nPoints"].astype(int).sum()
```

自动注入 semantic bug：

Mutation 1

Python

运行

```
.sum()  
→  
.mean()
```

Mutation 2

Python

运行

```
"ITA"  
→  
"ESP"
```

Mutation 3

删除 filter。

Mutation 4

Python

运行

>
→
>=

Mutation 5

替换 target column。

这些修改：

全部保证 Python 仍然可以成功运行。

然后专门测试 Auditor：

Can the auditor detect silent semantic mutations?

这会是一个非常漂亮的实验。

因为这样你终于不是只通过：

最终 answer wrong

来间接证明 Auditor 有用。

而是直接控制变量：

Original correct executable program
↓
Inject exactly one semantic mutation
↓
Still executable
↓
Can auditor locate the mutation?

然后测：

Detection Accuracy
Localization Accuracy
Repair Accuracy

而且分别报告：

这个实验我强烈建议做。

二十五、所以这两篇论文对你不是威胁，反而帮你补齐了三个关键点

第一，帮你找到了更准确的研究空隙：

Executability $\neq$ Semantic Faithfulness

第二，帮你把方法从一个“LLM Reviewer”升级成：

Question Intent IR + Static Code Intent + Dynamic Execution Trace + Table Grounding 的多粒度一致性审计。

第三，帮你找到一个非常强的实验：

Controlled Executable Semantic Mutation

专门制造：

“可以跑，但语义错”

然后检测你的方法能不能发现。

我现在最建议你的最终核心结构

我会把第二篇从之前的：

Multi-Granularity Semantic Logic Auditing

进一步收紧成：

Multi-Granularity Semantic Faithfulness Auditing for Executable Table Reasoning

核心公式思想就是：

$$\text{Execute}(P, T) \neq \text{Error} \not\Rightarrow P \models I(Q)$$

其中：

- Q : Question

- $I(Q)$: Question Intent
- P : generated Python program
- T : table

然后定义：

$$I_Q = (I_Q^G, I_Q^O, I_Q^P)$$

$$I_C = (I_C^G, I_C^O, I_C^P)$$

分别代表：

- Global
- Operator
- Parameter

最后：

$$Audit(I_Q, I_C, T, E)$$

其中 E 是 runtime execution trace。

这样你第二篇就不只是“代码逆向翻译”了，而是真正形成了一个结构化的 **semantic program verification** 框架。

我认为读完这两篇以后，最值得你立刻加到 **Codex** 里的不是再增加一个 **Agent**，而是三件事：**Static + Dynamic Code Intent**、**Controlled Semantic Mutation Tests**、**三级审计逐层消融**。这三个东西会明显增强第二篇的技术厚度。

来源